SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA0704

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools. *(BL4,BL5)*

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.
2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.
3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based

on overall link quality rather than just the shortest distance.Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

Q1. Network Throughput and Packet Loss Analysis

1. Problem Understanding

Inputs:

- Total data received = **8000 bits**
- Time = **4 seconds**
- Total packets sent = **200**
- Total packets received = **180**

Outputs:

- Network throughput in bits per second (bps)
- Packet loss percentage
- Network classification: **Efficient / Inefficient**
- Complete network performance summary

Constraints:

- Time must not be zero.
- Total packets sent must not be zero.
- Received packets should not exceed sent packets.

2. Formulas

Throughput:

Throughput = Total Data Received / Time

Packet Loss Percentage:

Packet Loss % = ((Packets Sent - Packets Received) / Packets Sent) × 100

Classification:

- If Throughput > 1500 **AND** Packet Loss < 10% → **Efficient**
- Otherwise → **Inefficient**

3. Pseudocode

```
ALGORITHM NetworkPerformanceAnalysis
BEGIN
   // Step 1: Input network data
   totalDataReceived ← 8000
   time ← 4
   packetsSent ← 200
   packetsReceived ← 180
   // Step 2: Validate input values
   IF time <= 0 THEN
      DISPLAY "Error: Time must be greater than zero."
      STOP
   END IF
   IF packetsSent <= 0 THEN
      DISPLAY "Error: Total packets sent must be greater than zero."
```

```
    STOP
END IF
IF packetsReceived < 0 OR packetsReceived > packetsSent THEN
    DISPLAY "Error: Invalid number of packets received."
    STOP
END IF
IF totalDataReceived < 0 THEN
    DISPLAY "Error: Total data received cannot be negative."
    STOP
END IF
// Step 3: Calculate throughput
throughput ← totalDataReceived / time
// Step 4: Calculate packet loss
lostPackets ← packetsSent - packetsReceived
packetLoss ← (lostPackets / packetsSent) × 100
// Step 5: Display calculated metrics
DISPLAY "========================================"
DISPLAY "      NETWORK PERFORMANCE SUMMARY"
DISPLAY "========================================"
DISPLAY "Total Data Received : ", totalDataReceived, " bits"
DISPLAY "Time                : ", time, " seconds"
DISPLAY "Packets Sent        : ", packetsSent
DISPLAY "Packets Received    : ", packetsReceived
DISPLAY "Packets Lost        : ", lostPackets
DISPLAY "Throughput          : ", throughput, " bps"
DISPLAY "Packet Loss         : ", packetLoss, "%"
// Step 6: Classify network
IF throughput > 1500 AND packetLoss < 10 THEN
    networkStatus ← "Efficient"
ELSE
    networkStatus ← "Inefficient"
END IF
// Step 7: Display final classification
DISPLAY "Network Status      : ", networkStatus
// Step 8: Final conclusion
IF networkStatus = "Efficient" THEN
    DISPLAY "Conclusion: The network is performing efficiently."
ELSE
    DISPLAY "Conclusion: The network is not performing efficiently."
```

```
        DISPLAY "Reason: Throughput must be greater than 1500 bps"
        DISPLAY "and packet loss must be less than 10%."
    END IF
END
```

4. Expected Output

```
========================================
    NETWORK PERFORMANCE SUMMARY
========================================
Total Data Received : 8000 bits
Time                : 4 seconds
Packets Sent        : 200
Packets Received    : 180
Packets Lost        : 20
Throughput          : 2000 bps
Packet Loss         : 10%
Network Status      : Inefficient
```

Conclusion: The network is not performing efficiently.

Reason: Throughput must be greater than 1500 bps and packet loss must be less than 10%.

5. Explanation

The algorithm first validates the input values to prevent invalid calculations such as division by zero. It then calculates **throughput** by dividing the total received data by the transmission time. Packet loss is calculated from the difference between packets sent and packets received. Finally, the algorithm checks both performance conditions: throughput must be **greater than 1500 bps** and packet loss must be **less than 10%**. Since the calculated packet loss is exactly **10%**, the second condition is not satisfied, so the network is classified as **Inefficient**.

Q2. Bandwidth Utilization Monitoring and Overloaded Link Detection

1. Problem Understanding

Inputs:

- Number of network links
- Bandwidth capacity of each link
- Current bandwidth usage of each link
- Backup link information
- Monitoring interval

Outputs:

- Bandwidth utilization of each link
- Link status
- Overloaded links
- Recommendation to switch traffic to a backup link

Constraint:

- If bandwidth utilization **exceeds 80%**, the link is considered overloaded.

2. Formula

For each network link:

Bandwidth Utilization (%)

Utilization = (Bandwidth Used / Total Bandwidth) × 100

3. Pseudocode

```
ALGORITHM BandwidthUtilizationMonitoring
BEGIN
   // Step 1: Initialize network links
   links ← ["Link1", "Link2", "Link3", "Link4"]
  totalBandwidth ← [100, 200, 150, 250]
   bandwidthUsed ← [60, 180, 135, 100]
   backupLinks ← ["Backup1", "Backup2", "Backup3", "Backup4"]
   numberOfLinks ← LENGTH(links)
   // Step 2: Continuously monitor all network links
   WHILE TRUE DO
      DISPLAY "========================================="
      DISPLAY "       BANDWIDTH MONITORING SYSTEM"
      DISPLAY "========================================="
      // Step 3: Check every network link
      FOR i ← 0 TO numberOfLinks - 1 DO
         // Calculate bandwidth utilization
         utilization ← (bandwidthUsed[i] /
                        totalBandwidth[i]) × 100
         DISPLAY "Link : ", links[i]
```

```
        DISPLAY "Total Bandwidth : ",
            totalBandwidth[i], " Mbps"
        DISPLAY "Bandwidth Used  : ",
            bandwidthUsed[i], " Mbps"
        DISPLAY "Utilization     : ",
            utilization, "%"
        // Step 4: Detect overloaded link
        IF utilization > 80 THEN
          DISPLAY "Status          : OVERLOADED"
          DISPLAY "Warning: ", links[i],
              " exceeds 80% utilization."
          // Step 5: Recommend backup link
          DISPLAY "Recommendation  : Switch traffic to ",
              backupLinks[i]
          DISPLAY "Traffic should be redirected ",
              "to reduce congestion."
        ELSE
          DISPLAY "Status          : NORMAL"
          DISPLAY "No traffic switching required."
        END IF
        DISPLAY "----------------------------------------"
      END FOR
      // Step 6: Wait before next monitoring cycle
      DISPLAY "Waiting for next monitoring interval..."
      WAIT 10 seconds
      // Step 7: Collect updated bandwidth usage
      FOR i ← 0 TO numberOfLinks - 1 DO
        bandwidthUsed[i] ← COLLECT_CURRENT_USAGE(links[i])
      END FOR
    END WHILE
END
```

4. Example Calculation

For **Link2**:

- Total bandwidth = 200 Mbps
- Bandwidth used = 180 Mbps

Utilization = (180 / 200) × 100

= 90%

Since **90% > 80%**, Link2 is classified as **OVERLOADED**.

The algorithm therefore recommends switching traffic to its corresponding **backup link**.

5. Expected Output

==========

BANDWIDTH MONITORING SYSTEM

==========

Link : Link1

Total Bandwidth : 100 Mbps

Bandwidth Used : 60 Mbps

Utilization : 60%

Status : NORMAL

No traffic switching required.

Link : Link2

Total Bandwidth : 200 Mbps

Bandwidth Used : 180 Mbps

Utilization : 90%

Status : OVERLOADED

Warning: Link2 exceeds 80% utilization.

Recommendation : Switch traffic to Backup2

Traffic should be redirected to reduce congestion.

Link : Link3

Total Bandwidth : 150 Mbps

Bandwidth Used : 135 Mbps

Utilization : 90%

Status : OVERLOADED

Warning: Link3 exceeds 80% utilization.

Recommendation : Switch traffic to Backup3

Link : Link4

Total Bandwidth : 250 Mbps

Bandwidth Used : 100 Mbps

Utilization : 40%

Status : NORMAL

No traffic switching required.

Waiting for next monitoring interval...

6. Explanation

The algorithm stores the bandwidth capacity and current usage of each network link in arrays. A continuous WHILE loop performs periodic monitoring, while a FOR loop checks every link individually. The utilization percentage is calculated for each link and compared with the **80% threshold**. If utilization exceeds 80%, the link is identified as overloaded and the algorithm recommends switching traffic to a backup link. After a fixed monitoring interval, updated bandwidth usage is collected and the process repeats.

Q3. Dynamic Best Communication Path Selection

1. Problem Understanding

Inputs:

- Network links between headquarters and six branch offices
- Bandwidth of each link
- Delay of each link
- Utilization of each link
- Link status: active/failed
- Available communication paths
- Monitoring interval

Outputs:

- Link-quality score for each path
- Best communication path
- Overloaded/failed links
- Alternate path when required
- Alert to the network administrator

Constraints:

- Higher bandwidth is preferred.
- Lower delay is preferred.
- Lower utilization is preferred.
- An overloaded or failed link should not be selected.
- The path with the highest valid quality score is selected.

2. Link Quality Score

A simple normalized scoring approach can be used:

Quality Score =

(Normalized Bandwidth × 0.5)

+ (Normalized Delay × 0.2)

+ (Normalized Utilization × 0.3)

For delay and utilization, lower values should produce better scores.

Therefore:

Bandwidth Score = Bandwidth / Maximum Bandwidth

Delay Score = Minimum Delay / Delay

Utilization Score = (100 - Utilization) / 100

This gives greater importance to bandwidth while also considering delay and current utilization.

3. Pseudocode

```
ALGORITHM DynamicBestPathSelection
BEGIN
    // Step 1: Initialize network information
    branches ← ["Branch1", "Branch2", "Branch3",
                "Branch4", "Branch5", "Branch6"]
    paths ← ["Path1", "Path2", "Path3",
             "Path4", "Path5", "Path6"]
    bandwidth ← [100, 80, 150, 120, 200, 90]
    delay ← [20, 30, 15, 25, 10, 35]
    utilization ← [50, 70, 60, 85, 40, 65]
    linkStatus ← ["ACTIVE", "ACTIVE", "ACTIVE",
                  "ACTIVE", "ACTIVE", "ACTIVE"]
    numberOfPaths ← LENGTH(paths)
    // Step 2: Continuously monitor the network
    WHILE TRUE DO
        DISPLAY "============================================="
        DISPLAY "       DYNAMIC PATH SELECTION SYSTEM"
        DISPLAY "============================================="
        // Step 3: Find maximum bandwidth
        maxBandwidth ← 0
        FOR i ← 0 TO numberOfPaths - 1 DO
            IF bandwidth[i] > maxBandwidth THEN
                maxBandwidth ← bandwidth[i]
            END IF
        END FOR
        // Step 4: Find minimum delay
        minDelay ← INFINITY
        FOR i ← 0 TO numberOfPaths - 1 DO
            IF linkStatus[i] = "ACTIVE" AND
               delay[i] < minDelay THEN
                minDelay ← delay[i]
            END IF
        END FOR
        // Step 5: Calculate quality score for each path
        FOR i ← 0 TO numberOfPaths - 1 DO
            IF linkStatus[i] = "FAILED" THEN
                qualityScore[i] ← 0
                DISPLAY paths[i], " : LINK FAILED"
```

```
    ELSE
        bandwidthScore ← bandwidth[i] / maxBandwidth
        delayScore ← minDelay / delay[i]
        utilizationScore ←
            (100 - utilization[i]) / 100
        qualityScore[i] ←
            (bandwidthScore × 0.5) +
            (delayScore × 0.2) +
            (utilizationScore × 0.3)
        DISPLAY "Path : ", paths[i]
        DISPLAY "Bandwidth : ", bandwidth[i], " Mbps"
        DISPLAY "Delay : ", delay[i], " ms"
        DISPLAY "Utilization : ", utilization[i], "%"
        DISPLAY "Quality Score : ",
                qualityScore[i]
    END IF
END FOR
// Step 6: Select the best valid path
bestPath ← "NONE"
highestScore ← -1
FOR i ← 0 TO numberOfPaths - 1 DO
    IF linkStatus[i] = "ACTIVE" AND
       utilization[i] <= 80 AND
       qualityScore[i] > highestScore THEN
        highestScore ← qualityScore[i]
        bestPath ← paths[i]
    END IF
END FOR
// Step 7: Check whether a best path exists
IF bestPath = "NONE" THEN
    DISPLAY "ALERT: No suitable communication path available."
    DISPLAY "Administrator action required."
ELSE
    DISPLAY "========================================="
    DISPLAY "BEST COMMUNICATION PATH"
    DISPLAY "========================================="
    DISPLAY "Selected Path : ", bestPath
    DISPLAY "Quality Score : ", highestScore
```

```
        END IF
        // Step 8: Detect overloaded or failed links
        FOR i ← 0 TO numberOfPaths - 1 DO
            IF linkStatus[i] = "FAILED" THEN
                DISPLAY "ALERT: ", paths[i],
                        " has failed."
                DISPLAY "Selecting an alternate path."
            ELSE IF utilization[i] > 80 THEN
                DISPLAY "ALERT: ", paths[i],
                        " is overloaded."
                DISPLAY "Utilization : ", utilization[i], "%"
                DISPLAY "This path will not be selected."
            END IF
        END FOR
        // Step 9: Collect updated network metrics
        WAIT 10 seconds
        FOR i ← 0 TO numberOfPaths - 1 DO
            bandwidth[i] ←
                COLLECT_CURRENT_BANDWIDTH(paths[i])
            delay[i] ←
                COLLECT_CURRENT_DELAY(paths[i])
            utilization[i] ←
                COLLECT_CURRENT_UTILIZATION(paths[i])
            linkStatus[i] ←
                CHECK_LINK_STATUS(paths[i])
        END FOR
    END WHILE
END
```

4. Example

Suppose the monitoring system obtains:

Path	Bandwidth	Delay	Utilization	Status
Path1	100 Mbps	20 ms	50%	Active
Path2	80 Mbps	30 ms	70%	Active
Path3	150 Mbps	15 ms	60%	Active
Path4	120 Mbps	25 ms	85%	Active
Path5	200 Mbps	10 ms	40%	Active
Path6	90 Mbps	35 ms	65%	Active

Path4 has 85% utilization, so it is treated as **overloaded** and excluded from selection.

The algorithm calculates the quality score of the remaining paths and selects the path having the

highest quality score, rather than simply selecting the path with the shortest distance.

5. Expected Output

```
==========================================
      DYNAMIC PATH SELECTION SYSTEM
==========================================
Path : Path1
Bandwidth : 100 Mbps
Delay : 20 ms
Utilization : 50%
Quality Score : 0.670
Path : Path2
Bandwidth : 80 Mbps
Delay : 30 ms
Utilization : 70%
Quality Score : 0.443
Path : Path3
Bandwidth : 150 Mbps
Delay : 15 ms
Utilization : 60%
Quality Score : 0.740
Path : Path4
Bandwidth : 120 Mbps
Delay : 25 ms
Utilization : 85%
Quality Score : 0.577
Path : Path5
Bandwidth : 200 Mbps
Delay : 10 ms
Utilization : 40%
Quality Score : 0.940
Path : Path6
Bandwidth : 90 Mbps
Delay : 35 ms
Utilization : 65%
Quality Score : 0.395
ALERT: Path4 is overloaded.
Utilization : 85%
This path will not be selected.
==========================================
```

BEST COMMUNICATION PATH

==========

Selected Path : Path5

Quality Score : 0.940

6. Explanation

The algorithm continuously monitors the available communication paths and stores **bandwidth, delay, utilization, and link status** in arrays. It calculates a quality score for every active path by considering all three network-performance metrics instead of relying only on distance. A path with utilization above **80%** is considered overloaded and is excluded from selection. If a link fails or becomes overloaded, the algorithm automatically searches for another valid path and generates an alert for the administrator. The monitoring process then repeats at regular intervals using updated network measurements.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient